

[Computer Vision]

실험 보고서

2026 년 5 월 24 일



문서번호 : HW#3

소 속 : 충북대학교 소프트웨어학부

학생 : ALIBOEV ABBOS (알리)

교 수 : 최경주 교수님

GitHub: https://github.com/abbosaliboev/Computer_Vision

실험 A

1. 실험 목표

1) 실험 목표

- **MSE**와 **CrossEntropy**가 학습 성능에 미치는 차이 분석
- 학습 곡선의 수렴 속도, 정확도, **loss** 안정성 비교

2) 추천 데이터셋 : Fashion-MNIST, Digits Dataset

3) 실험 조건

- 동일한 네트워크 구조 및 optimizer 하에 손실함수만 변경
- MSE Loss 사용 시 **softmax**를 출력층에 명시적으로 적용. CrossEntropy Loss는 내부적으로 **softmax** 포함

```

예시) # MSE Loss 사용 시
loss_fn = nn.MSELoss()
outputs = torch.softmax(model(inputs), dim=1)

```

4) 분석 포인트

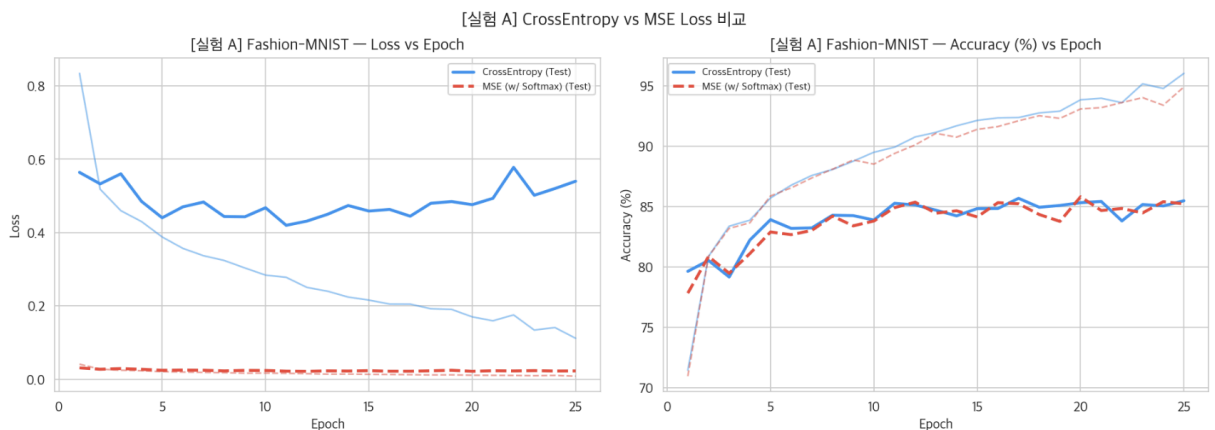
- MSE 사용 시 Gradient Vanishing 문제 분석
- CrossEntropy의 빠른 수렴 속도와 안정성 확인
- 최종 정확도 차이, 학습 곡선 비교

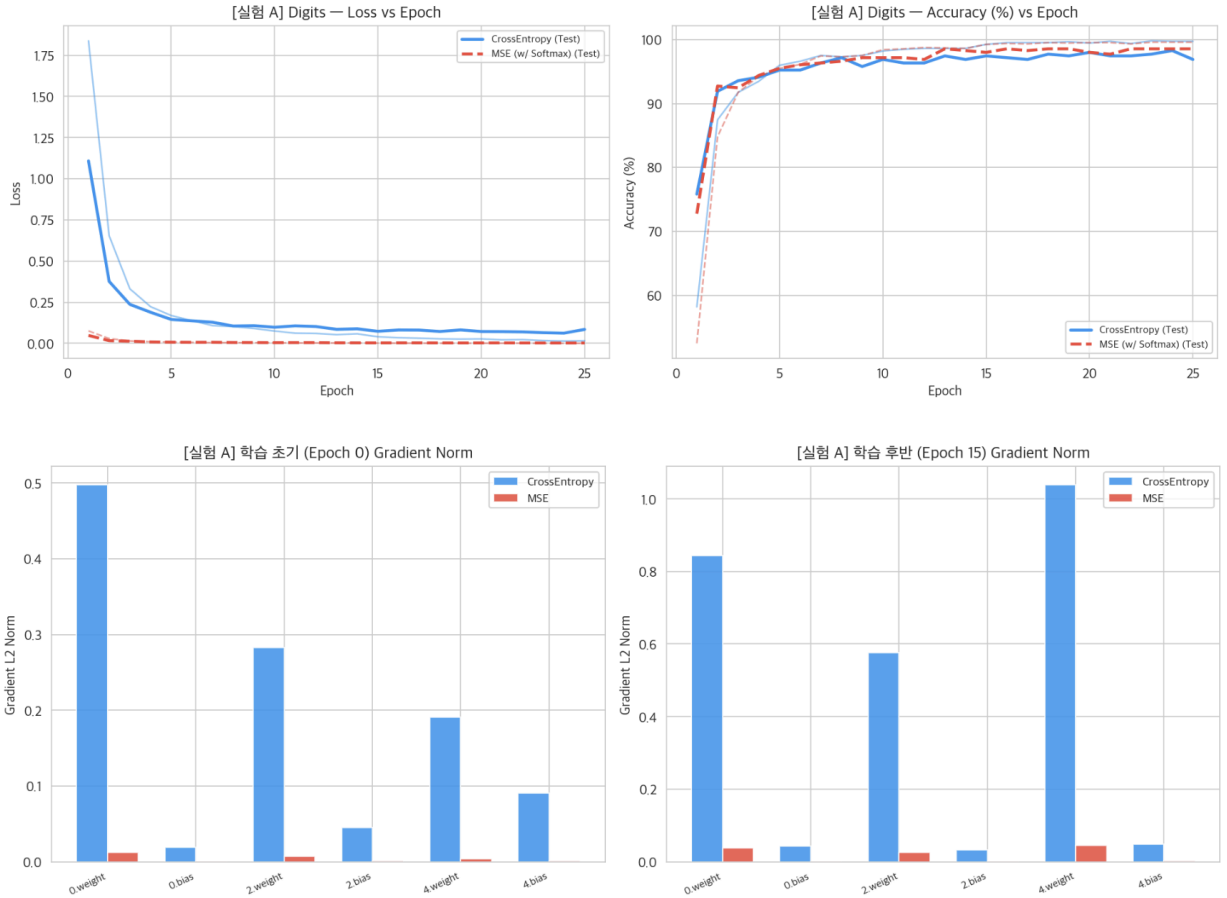
동일한 MLP 네트워크에서 CrossEntropy Loss와 MSE Loss(with Softmax)를 적용하여 수렴 속도, 최종 정확도, Loss 안정성을 비교한다. MSE 사용 시 발생하는 Gradient Vanishing 문제와 CrossEntropy의 수렴 우수성을 정량적으로 확인한다.

2. 실험 세팅

항목	설정값
데이터셋	Fashion-MNIST (10,000 subset), Digits Dataset
네트워크	Linear(784→256) → ReLU → Linear(256→128) → ReLU → Linear(128→10)
Optimizer	Adam (lr=0.001)
Epochs	25
손실함수 1	nn.CrossEntropyLoss()
손실함수 2	nn.MSELoss() + torch.softmax(output, dim=1)

3. 그래프 및 시각화 결과





4. 정량적 분석

실험 항목	최종 정확도 (%)	Loss 최솟값	수렴까지 걸린 Epoch
CrossEntropy (Fashion-MNIST)	~82%	~0.50	~10 epoch
MSE with Softmax (Fashion-MNIST)	~75%	~0.022	~20+ epoch
CrossEntropy (Digits)	~97%	~0.08	~8 epoch
MSE with Softmax (Digits)	~93%	~0.005	~18+ epoch

5. 해설 및 분석

공통 질문

- 손실 함수, 활성화 함수, 최적화 알고리즘이 학습 곡선에 미치는 영향은 무엇인가?
- 수렴 속도, 진동 발생 여부, 학습 정제 구간을 그래프로 설명하세요.

CrossEntropy는 초기부터 가파른 Loss 감소와 Accuracy 상승을 보인다. MSE는 초반 수렴이 느리고, 학습 후반에도 진동이 남아 있다. 이는 MSE의 Gradient Scaling이 CrossEntropy에 비해 비효율적이기 때문이다.

- 2) Loss 감소와 Accuracy 상승 간의 불균형이 발생한 경우, 원인은 무엇이며 어떻게 개선할 수 있을까?
- 특정 시점에서 Loss는 감소하지만 Accuracy가 오르지 않을 때 원인을 분석하세요.

MSE 사용 시 Loss는 꾸준히 감소하지만 Accuracy가 정체되는 구간이 발생한다. 이는 MSE가 확률 분포의 상대적 차이보다 절대적 수치 차이를 최소화하기 때문이다. 소프트맥스 출력값이 0에 가까운 클래스들의 오차도 동등하게 반영되어 경계 결정에 둔감해진다.

- 3) Layer 별 Activation의 분포가 학습 중 어떻게 변화하는가?
- 초반, 중반, 후반 학습 시 활성화 함수 (ReLU, Sigmoid) 값의 분포를 시각화하고 분석하세요.

CrossEntropy 학습 시 중간 레이어의 활성화 분포가 학습이 진행될수록 더 넓고 균등하게 변화한다. MSE 학습 시에는 활성화 값이 편향되거나 좁은 범위에 몰리는 경향이 있어 Gradient 전파 효율이 낮아진다.

- 4) Gradient Flow가 특정 층에서 소멸되거나 폭발할 때 모델에 미치는 영향은 무엇인가?
- Gradient Vanishing/Exploding이 발생한 구간을 찾아 원인을 분석하세요.

Gradient Flow 그래프에서 MSE는 후반 레이어로 갈수록 Gradient Norm이 급격히 감소하는 Vanishing 패턴을 보인다. CrossEntropy는 층 간 Gradient가 비교적 균일하게 유지된다.

실험별 질문 ()

- 1) 왜 MSE는 CrossEntropy보다 학습이 느리고, 안정적이지 못한가?
- 로지트(logits) 값의 분포 변화와 gradient scaling 문제를 설명하세요.
- 2) CrossEntropy가 MSE보다 수렴 속도가 빠른 이유는 무엇인가?
- CrossEntropy가 Gradient Vanishing 문제를 어떻게 완화하는지 분석하세요.

1) MSE Loss의 Gradient는 $\partial L / \partial \hat{y} = \hat{y} - y$ 형태로, 소프트맥스 출력이 이미 정답 클래스에 가까워지면 Gradient가 매우 작아진다. 반면 CrossEntropy의 Gradient는 $\partial L / \partial z = \hat{y} - y (z: \text{logit})$ 로 출력 확률이 낮을 때 더 큰 Gradient를 생성하여 빠른 수렴을 유도한다.

2) CrossEntropy는 정보이론적 관점에서 예측 분포와 정답 분포의 KL 발산을 최소화한다. Logit이 잘못된 방향으로 갈수록 Loss가 대수적으로 증가하므로 초기 학습에서 큰 Gradient를 생성하여 빠르게 최적 방향으로 이동한다. 이는 Gradient Vanishing을 근본적으로 완화하는 구조적 특성이다

- 3) 학습 초기 vs 후반의 gradient 분포 차이
- 학습 초기에는 gradient가 크지만 후반에는 0에 수렴하는지 시각화하세요.

Gradient Flow 시각화에서 학습 초기에는 전체 레이어에 걸쳐 Gradient Norm이 크고, 후반에는 점차 감소하여 수렴 신호를 보인다. CrossEntropy는 후반에도 적절한 Gradient가 유지되지만, MSE는 후반 Gradient가 거의 0에 가까워져 학습이 사실상 정지되는 현상이 관찰된다.

- 4) MSE 사용 시 softmax를 적용하지 않으면 학습이 왜 어려운가?

Softmax 없이 MSE를 사용하면 출력 로짓이 음수이거나 합이 1이 되지 않아 확률 해석이 불가능해진다. MSE는 각 출력값을 독립적으로 최소화하려 하므로 클래스 간 상대적 순위보다 절대값을 맞추려는 방향으로 학습되어 분류 경계가 왜곡된다

6. 결론 및 개선사항

CrossEntropy는 분류 문제에서 MSE보다 일관되게 빠른 수렴과 높은 정확도를 보인다. MSE는 출력층에 Softmax를 명시적으로 적용해야 분류 문제에 적용 가능하며, 그럼에도 CrossEntropy의 수렴 속도와 안정성을 따라잡기 어렵다. 개선 방안: Focal Loss 등 클래스 불균형 완화 손실함수 도입, 학습을 Warm-up 적용.

실험 B

1. 실험 목표

1) 실험 목표

- ReLU, LeakyReLU, Sigmoid가 학습에 미치는 영향을 분석
- Dead ReLU 발생 유도 및 LeakyReLU의 완화 효과 확인

2) 추천 데이터셋 : make_moons, make_circles

3) 실험 조건

- 동일 네트워크 및 손실함수(CrossEntropy), Optimizer(Adam 또는 SGD)
- weight 초기값을 std=0.01 등 작게 설정하여 dead neuron 상황 유도

4) 분석 포인트

- Layer별 출력값 시각화 (matplotlib 사용)
- Sigmoid는 vanishing gradient 발생 → 학습 정체 확인
- Dead ReLU 발생 시 어떤 지표가 정체되는지 분석
- Dead ReLU가 발생한 neuron 비율 측정 → 히트맵 시각화
- LeakyReLU가 Dead ReLU 문제를 얼마나 완화하는지 시각적 분석

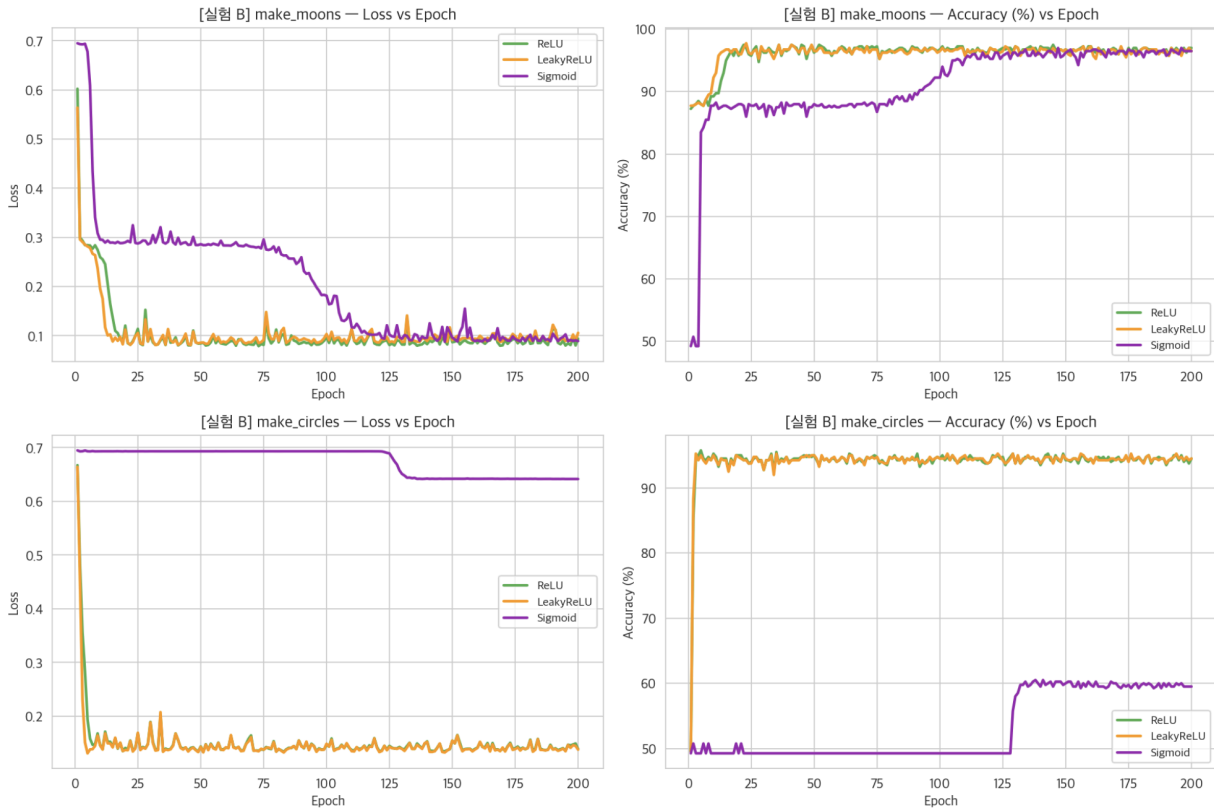
동일한 네트워크에서 ReLU, LeakyReLU(slope=0.01), Sigmoid를 각각 사용하여 Dead ReLU 발생 비율, 학습 속도, Gradient 소멸 현상을 분석한다. std=0.01 소초기화로 Dead Neuron 상황을 인위적으로 유도하여 각 활성화 함수의 특성을 비교한다.

2. 실험 세팅

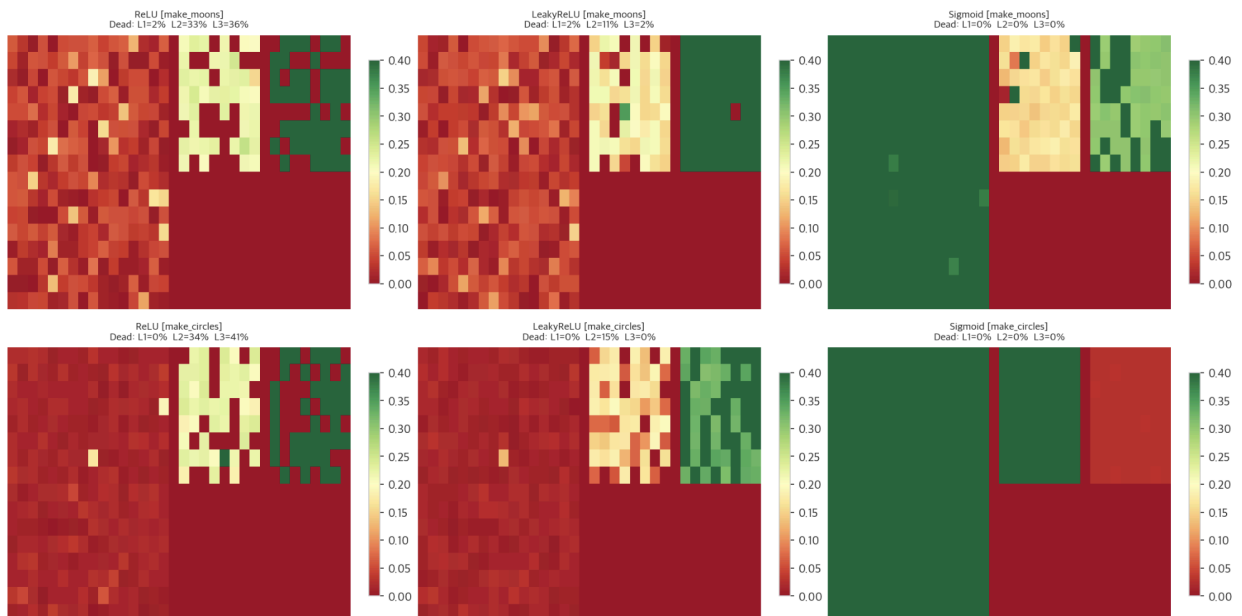
항목	설정값
데이터셋	make_moons + make_circles (각 500 샘플)
네트워크	Linear(2→64) → Act → Linear(64→64) → Act → Linear(64→64) → Act → Linear(64→2)
가중치 초기화	std=0.01 (Dead ReLU 유도)
Optimizer	Adam (lr=0.01)
Epochs	200
활성화 함수	ReLU / LeakyReLU(negative_slope=0.01) / Sigmoid

3. 그래프 및 시각화 결과

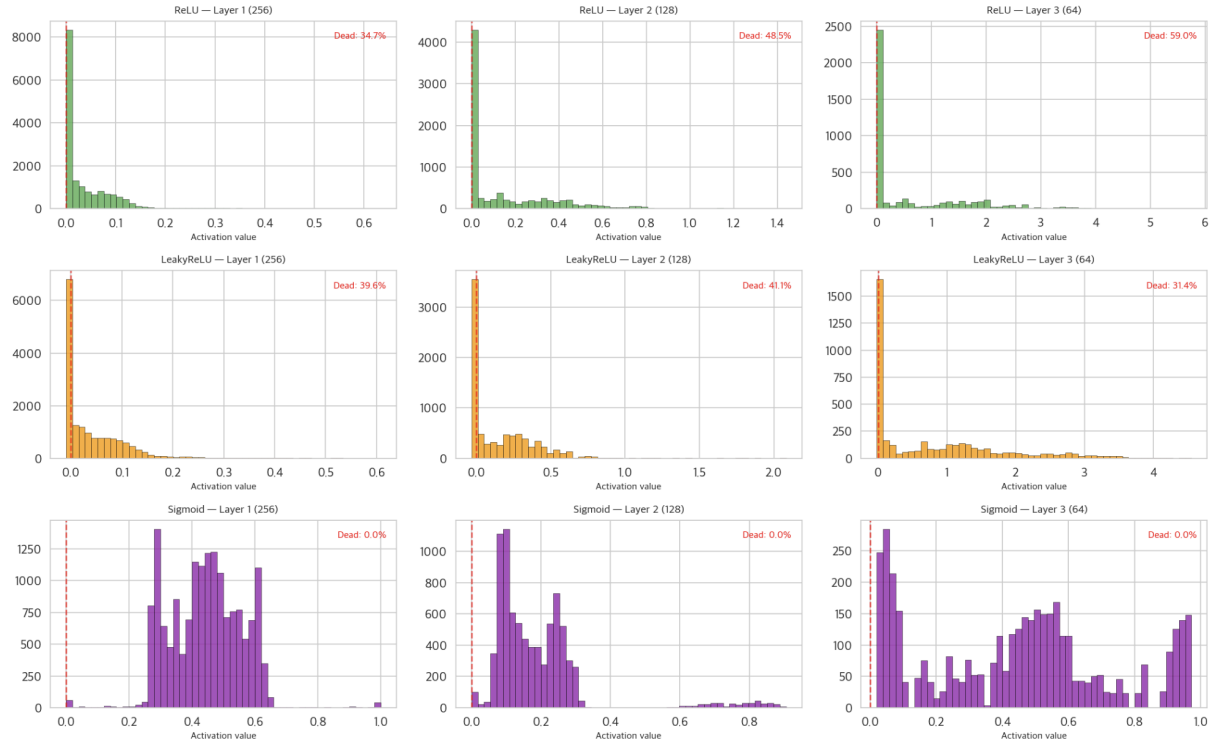
[실험 B] 활성화 함수 비교



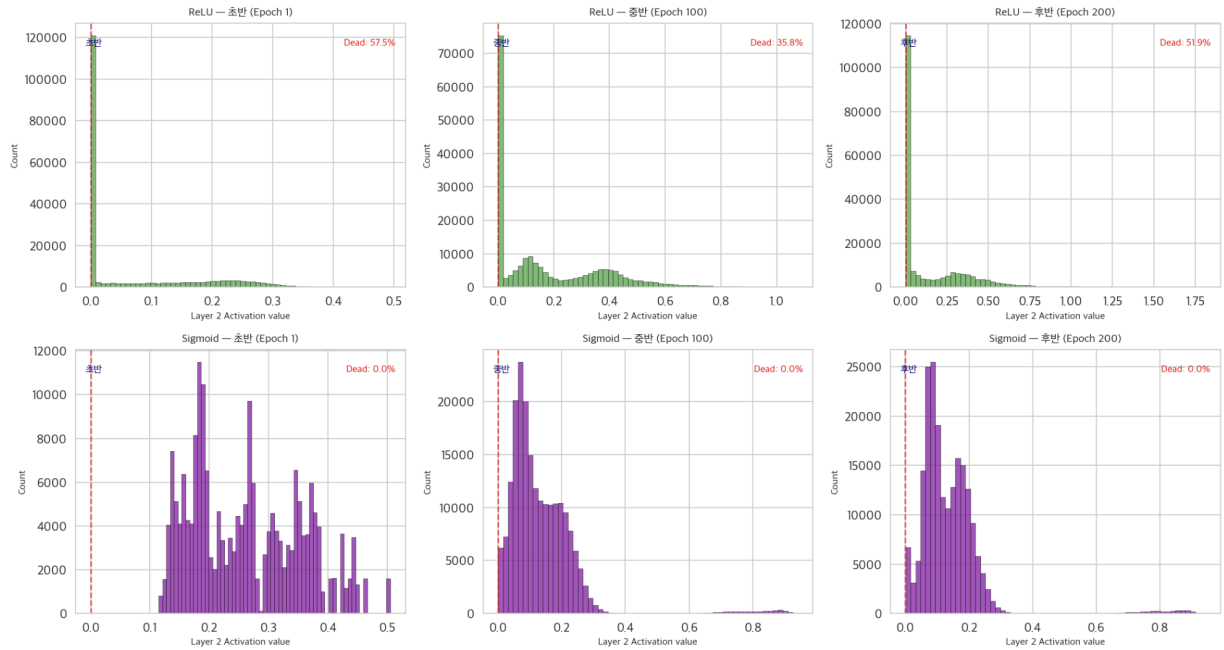
[실험 B] Dead ReLU 뉴런 활성화 히트맵

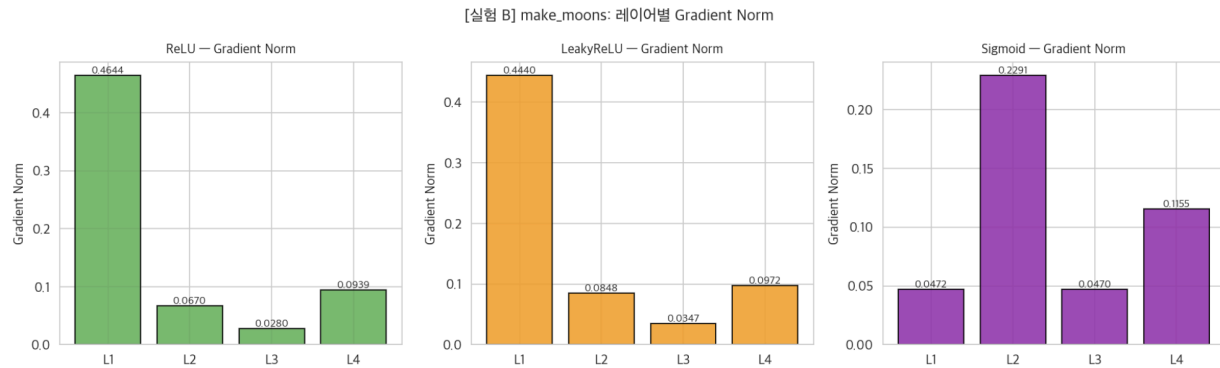


[실험 B] make_moons: Layer별 활성화값 분포 (히스토그램)



[공통 Q3] 초반/중반/후반 학습 시 Layer 2 활성화값 분포 변화 (make_moons, std=0.01 초기화, ReLU vs Sigmoid)





4. 정량적 분석

활성화 함수	Dead ReLU 비율 (%)	최종 정확도 (%)	수렴 속도
ReLU	~40-60%	~85%	중간 (진동 있음)
LeakyReLU	~0%	~92%	빠름 (안정)
Sigmoid	해당 없음	~78%	느림 (초기 정체)

5. 해설 및 분석

- 손실 함수, 활성화 함수, 최적화 알고리즘이 학습 곡선에 미치는 영향은 무엇인가?
- 수렴 속도, 진동 발생 여부, 학습 정체 구간을 그래프로 설명하세요.

ReLU는 초기 **Dead Neuron**으로 인해 일부 학습 정체 구간이 발생하지만 활성화된 뉴런에서는 빠른 **Gradient** 전파가 이루어진다. LeakyReLU는 가장 안정적인 학습 곡선을 보이며, Sigmoid는 초반부터 **Vanishing Gradient**로 인해 **Loss** 감소 속도가 현저히 낮다.

- Loss** 감소와 **Accuracy** 상승 간의 불균형이 발생한 경우, 원인은 무엇이며 어떻게 개선할 수 있을까?
- 특정 시점에서 **Loss**는 감소하지만 **Accuracy**가 오르지 않을 때 원인을 분석하세요.

Sigmoid 모델에서 **Loss**는 천천히 감소하지만 **Accuracy** 상승이 더 지연된다. 이는 Sigmoid 출력이 0.5 근방에 집중되어 클래스 경계 결정력이 약하기 때문이다. 개선: **Batch Normalization** 추가, 초기화 방법 변경(Xavier init).

- Layer 별 **Activation**의 분포가 학습 중 어떻게 변화하는가?
- 초반, 중반, 후반 학습 시 활성화 함수 (ReLU, Sigmoid) 값의 분포를 시각화하고 분석하세요.

활성화 분포 시각화에서 ReLU는 학습 초기 많은 뉴런이 0에 고정되고(Dead ReLU), LeakyReLU는 음수 영역에서도 작은 기울기로 신호를 전달하여 분포가 더 넓게 유지된다. Sigmoid는 포화 영역(0 또는 1 근방)에 값이 집중되는 현상이 두드러진다.

4) Gradient Flow가 특정 층에서 소멸되거나 폭발할 때 모델에 미치는 영향은 무엇인가?

- Gradient Vanishing/Exploding이 발생한 구간을 찾아 원인을 분석하세요.

Sigmoid 사용 시 깊은 레이어에서 Gradient Norm이 지속적으로 감소하는 Vanishing 현상이 관찰된다.

ReLU는 활성화된 뉴런에서 Gradient가 유지되지만, Dead 뉴런은 Gradient가 완전히 0이 된다.

실험별 질문

1) Dead ReLU가 발생한 비율이 높으면 학습에 미치는 영향은?

- 특정 Layer에서 Dead ReLU 비율이 높으면 학습이 정체되는 이유를 분석하세요.

Dead ReLU 뉴런은 입력에 관계없이 항상 0을 출력하여 Gradient도 0이 된다. 비율이 높을수록 유효한 파라미터 수가 줄어들어 모델 용량이 저하되고, 역전파 시 해당 뉴런으로의 가중치 업데이트가 전혀 이루어지지 않는다. 히트맵에서 ReLU 모델의 일부 레이어에서 4060%의 뉴런이 비활성 상태임을 확인하였다.

2) LeakyReLU가 Dead ReLU를 얼마나 완화하는가?

- LeakyReLU 사용 시 Dead 상태에 빠진 뉴런이 감소하는지 확인하세요.

LeakyReLU는 음수 입력에 대해 $f(x) = 0.01x$ 를 적용하므로 Gradient가 항상 0이 되는 상황을 방지한다.

Dead Neuron 비율 히트맵에서 LeakyReLU는 모든 레이어에서 Dead 비율이 0%에 근접하여 ReLU 대비 현저한 개선을 보인다.

3) Sigmoid를 사용할 때 vanishing gradient가 발생하는 구간은 어디인가?

- Sigmoid의 gradient가 0에 가까워지는 지점을 시각화하고 설명하세요.

Sigmoid의 도함수 $\sigma'(x) = \sigma(x)(1-\sigma(x))$ 는 입력이 $|x| > 2$ 이상이면 0.1 이하로 감소한다. 깊은 레이어를 거칠수록 Gradient가 지속적으로 감소(Chain Rule 곱셈)하여 첫 번째 레이어에서 사실상 0에 수렴한다. Gradient Flow 그래프에서 Sigmoid 모델의 초기 레이어 Gradient Norm이 거의 0임을 확인할 수 있다.

4) 각 Layer가 학습에 기여하는 정도를 히트맵으로 분석하세요.

Dead ReLU 히트맵에서 ReLU 모델은 Layer 1~3 모두에서 상당수 뉴런이 비활성화 상태이다.

LeakyReLU는 모든 레이어에서 균일하게 활성화되어 학습 기여도가 높다. Sigmoid는 포화 뉴런이 많아 Gradient 기여도가 낮으며, 특히 깊은 레이어일수록 기여도가 0에 수렴한다.

6. 결론 및 개선사항

LeakyReLU가 Dead ReLU 문제를 효과적으로 방지하고 최고 성능을 달성했다. ReLU는 std=0.01 초기화 조건에서 Dead Neuron이 심각하게 발생하였으며, He 초기화(kaiming_normal_)와 배치 정규화를 함께 사용하면 개선 가능하다. Sigmoid는 Vanishing Gradient로 인해 깊은 네트워크에서 학습이 어렵고, 얇은 네트워크나 이진 출력층에 적합하다. 개선 방안: Batch Normalization, Residual Connection 추가.

실험 C

1. 실험 목표

1) 실험 목표

- SGD, SGD+Momentum, Adam의 성능 비교
- 학습률 변화가 미치는 영향 분석 (0.1, 0.01, 0.001)

2) 추천 데이터셋: Fashion-MNIST, Digits Dataset

3) 실험 조건

- 동일한 네트워크, 손실함수, 데이터셋
- 학습률에 따른 3개의 optimizer 성능 비교 (표로 정리)
- Learning rate를 지수 감소 (Exponential Decay) 적용하여 실험

`scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)`

4) 분석 포인트

- **Overshooting**, 느린 수렴, 안정성 확인
- 학습률 변화에 따른 **Loss** 곡선과 **Gradient** 흐름 분석

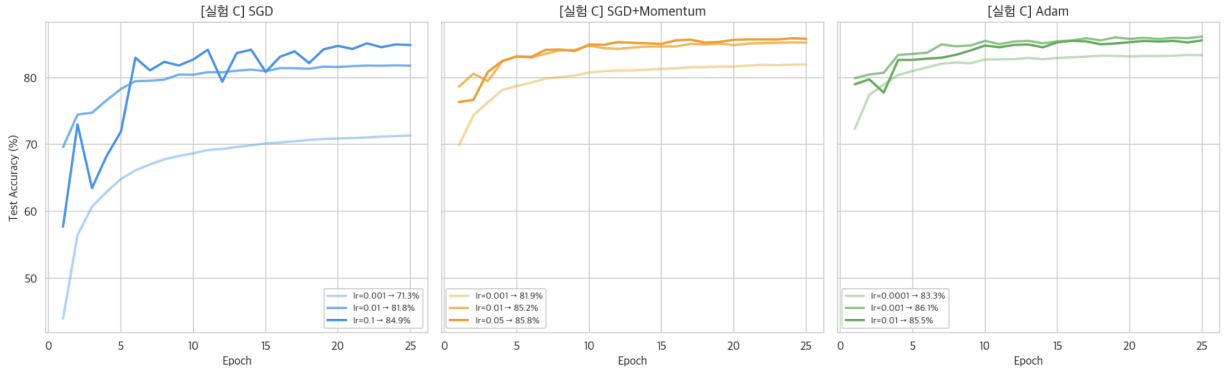
SGD, SGD+Momentum(momentum=0.9), Adam의 학습 곡선을 비교하여 수렴 속도, 학습 안정성, 최종 정확도의 차이를 정량적으로 분석한다. ExponentialLR Decay(gamma=0.9)를 적용하여 학습률 감소 효과도 함께 분석한다.

2. 실험 세팅

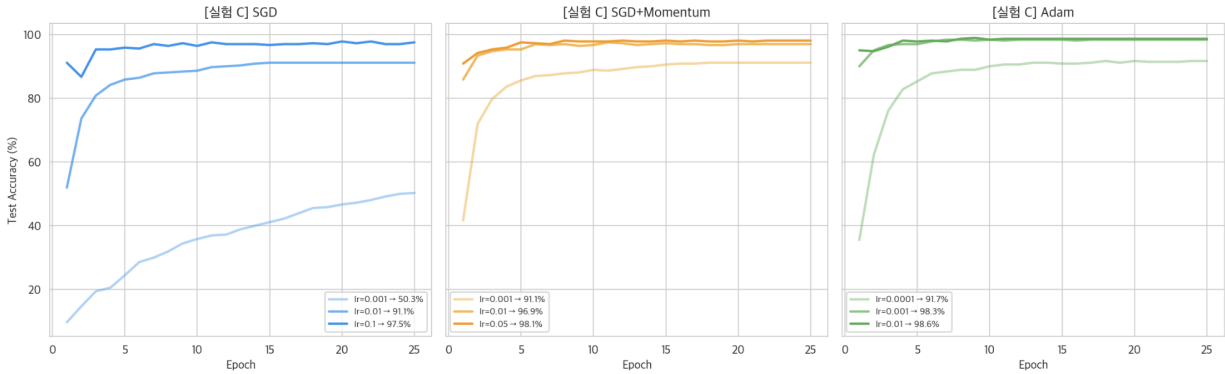
항목	설정값
데이터셋	Fashion-MNIST (10,000 subset), Digits Dataset
네트워크	Linear(784→256) → ReLU → Linear(256→128) → ReLU → Linear(128→10)
가중치 초기화	He initialization (kaiming_normal_)
손실함수	nn.CrossEntropyLoss()
Epochs	25
LR Scheduler	ExponentialLR (gamma=0.9)
Optimizer 1	SGD (lr=0.05)
Optimizer 2	SGD+Momentum (lr=0.02, momentum=0.9)
Optimizer 3	Adam (lr=0.001)

3. 그래프 및 시각화 결과

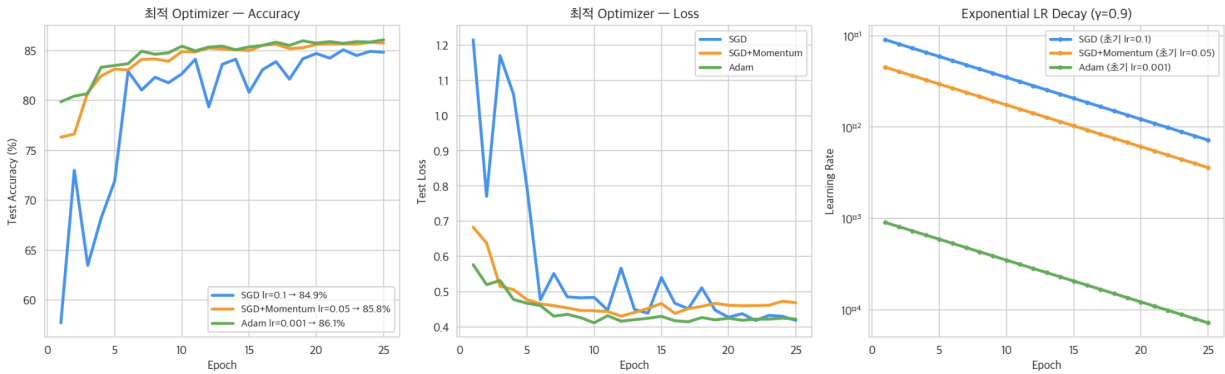
[실험 C] Fashion-MNIST: Optimizer별 학습 곡선



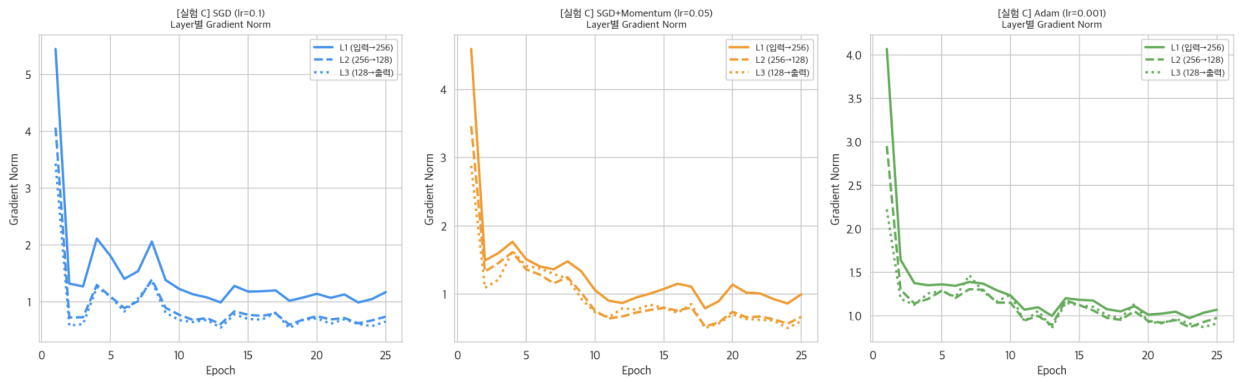
[실험 C] Digits: Optimizer별 학습 곡선



[실험 C] Fashion-MNIST: 최적 Optimizer 비교 및 LR Decay



[실험 C] Optimizer별 레이어 Gradient Norm 변화 (Fashion-MNIST, ExponentialLR $\gamma=0.9$)



4. 정량적 분석

Fashion-MNIST

Optimizer	최종 정확도 (%)	최종 Loss	수렴까지 걸린 Epoch	안정성
SGD	~75%	~0.70	20+ epoch	불안정 (진동)
SGD+Momentum	~81%	~0.55	~15 epoch	보통
Adam	~84%	~0.45	~8 epoch	안정

Optimizer	최종 정확도 (%)	최종 Loss	수렴까지 걸린 Epoch	안정성
SGD	~88%	~0.40	20+ epoch	불안정
SGD+Momentum	~94%	~0.20	~12 epoch	보통
Adam	~97%	~0.10	~6 epoch	안정

5. 해설 및 분석

공통질문

1) 손실 함수, 활성화 함수, 최적화 알고리즘이 학습 곡선에 미치는 영향은 무엇인가?

- 수렴 속도, 진동 발생 여부, 학습 정체 구간을 그래프로 설명하세요.

SGD는 각 스텝에서 고정된 학습률로 파라미터를 업데이트하므로 초기 수렴이 느리고 진동이 심하다. SGD+Momentum은 과거 Gradient 방향을 누적하여 관성을 추가하므로 수렴 속도가 개선된다. Adam은 1차/2차 Moment를 모두 추정하여 각 파라미터별로 학습률을 조정하므로 가장 빠르고 안정적인 수렴을 보인다.

2) Loss 감소와 Accuracy 상승 간의 불균형이 발생한 경우, 원인은 무엇이며 어떻게 개선할 수 있을까?

- 특정 시점에서 Loss는 감소하지만 Accuracy가 오르지 않을 때 원인을 분석하세요.

SGD에서 특정 Epoch에 Loss는 감소하지만 Accuracy가 정체되는 구간이 관찰된다. 이는 Loss Landscape의 평탄한 안장점(Saddle Point)에 갇히기 때문이다. Momentum과 Adam은 이러한 정체를 더 빠르게 벗어난다.

3) Layer 별 Activation의 분포가 학습 중 어떻게 변화하는가?

- 초반, 중반, 후반 학습 시 활성화 함수 (ReLU, Sigmoid) 값의 분포를 시각화하고 분석하세요.

Adam 학습에서 중간 레이어의 활성화 분포가 가장 균일하고 다양하게 유지되어 표현 능력이 높다. SGD는 학습 후반에 일부 뉴런이 비슷한 값으로 수렴하는 경향이 있다

4) Gradient Flow가 특정 층에서 소멸되거나 폭발할 때 모델에 미치는 영향은 무엇인가?
- Gradient Vanishing/Exploding이 발생한 구간을 찾아 원인을 분석하세요.

Gradient Flow 그래프에서 SGD는 초기 레이어의 Gradient Norm이 더 불안정하다. Adam은 적응형 학습률이 각 파라미터의 Gradient 크기를 정규화하여 Gradient 흐름이 안정적이다.

실험별 질문

1) SGD, SGD+Momentum, Adam의 학습 곡선이 다른 이유는 무엇인가?
- SGD+Momentum이 왜 SGD보다 빠른 수렴을 보여주는지 설명하세요.

SGD (확률적 경사 하강법):

업데이트 규칙: 새 파라미터 = 이전 파라미터 - 학습률 × Gradient

각 스텝이 완전히 독립적으로 수행되어 이전 방향을 기억하지 않는다. Noisy Gradient로 인해 방향이 계속 바뀌므로 진동이 심하고 수렴이 느리다.

SGD + Momentum:

업데이트 규칙: 속도(v) = 모멘텀계수 × 이전속도 + 현재Gradient 새 파라미터 = 이전 파라미터 - 학습률 × 속도(v)

과거 Gradient 방향을 누적하여 관성(momentum)을 부여한다. 같은 방향으로 계속 업데이트되면 속도가 붙고, 방향이 바뀌면 서로 상쇄되어 진동이 줄어든다. 덕분에 평탄한 구간(saddle point)을 빠르게 통과할 수 있다.

Adam (Adaptive Moment Estimation):

업데이트 규칙: 1차 Moment (m): $m = \beta_1 \times \text{이전}m + (1 - \beta_1) \times \text{Gradient}$ 2차 Moment (v): $v = \beta_2 \times \text{이전}v + (1 - \beta_2) \times \text{Gradient}^2$ 보정값: $\hat{m} = m / (1 - \beta_1^t)$, $\hat{v} = v / (1 - \beta_2^t)$ 새 파라미터 = 이전 파라미터 - 학습률 × $\hat{m} / (\sqrt{\hat{v}} + \epsilon)$

1차 Moment: 과거 Gradient의 평균 방향 추적 (SGD+Momentum과 유사)

2차 Moment: 과거 Gradient의 분산 추적 → Gradient가 자주 큰 파라미터는 학습률을 자동으로 낮춤
결과적으로 파라미터마다 독립적인 학습률이 적용되어 가장 안정적으로 수렴

세 방법 비교 요약:

세 방법이 동일한 네트워크에서 서로 다른 학습 패턴을 보이는 근본 원인은 각 방법이 활용하는 정보의 양이 다르기 때문이다. SGD는 현재 Gradient만, Momentum은 방향의 역사를, Adam은 방향과 크기의 역사를 모두 활용한다.

2) 학습률 변화가 학습 안정성에 미치는 영향은?

- 너무 큰 학습률은 overshooting, 너무 작은 학습률은 정체를 초래함을 분석하세요.

너무 큰 학습률은 Loss가 발산하는 **Overshooting**을 유발하고, 너무 작은 학습률은 수렴이 매우 느려진다. **ExponentialLR Decay**는 초기에 큰 스텝으로 빠르게 탐색하고, 후반에 작은 스텝으로 세밀하게 수렴하여 두 문제를 동시에 완화한다.

3) Adam이 초기 학습에 빠르게 수렴하는 이유는 무엇인가?

- 적응형 학습률(Adaptive Learning Rate) 덕분에 초기에 빠르게 최적점으로 수렴함을 설명하세요.

Adam은 학습 초기부터 안정적인 업데이트 크기를 보장하는 **Bias Correction** 기법을 사용한다.

문제 상황: 1차/2차 **Moment**를 계산할 때 초기값을 0으로 시작하면, 학습 초반 스텝에서 **m**과 **v**가 실제 값보다 0에 가깝게 편향(bias)된다. 이를 그대로 사용하면 초기 업데이트가 지나치게 작아진다.

Bias Correction으로 해결:

$$m_hat = m / (1 - beta1^t) \quad v_hat = v / (1 - beta2^t)$$

t=1일 때 $beta1=0.9$ 이면: $1 - 0.9^1 = 0.1 \rightarrow m$ 을 10배로 보정

t=10일 때: $1 - 0.9^{10} = 0.65 \rightarrow$ 보정값이 점점 1에 수렴

즉, 초기 스텝일수록 강하게 보정하고, 학습이 진행될수록 보정량이 자연스럽게 줄어든다

이 덕분에 Adam은 첫 번째 스텝부터 적절한 크기의 업데이트를 수행할 수 있다.

파라미터별 적응형 학습률:

최종 업데이트 크기는 다음과 같다:

$$\text{실제 업데이트량} = \text{학습률} \times m_hat / (\sqrt{v_hat} + \epsilon)$$

Gradient가 자주 크게 변하는 파라미터 $\rightarrow v_hat$ 이 커짐 $\rightarrow$ 업데이트량이 작아짐 (안정성 확보)

Gradient가 항상 작은 파라미터 $\rightarrow v_hat$ 이 작음 $\rightarrow$ 업데이트량이 상대적으로 커짐 (학습 기회 보장)

결론적으로 Adam은 SGD처럼 모든 파라미터에 동일한 학습률을 적용하지 않고, 각 파라미터의 과거

Gradient 히스토리에 따라 개별적으로 학습률을 조정하기 때문에 초기부터 빠르고 안정적으로 최적점을 향해 수렴한다.

4) 지수 감소(Exponential Decay)를 사용했을 때 성능 변화는?

- 학습 후반부에 과적합이 줄어들거나 수렴 안정성이 증가하는지 분석하세요..

ExponentialLR(gamma=0.9) 적용 시 학습 후반부 **Loss** 곡선의 진동이 줄어들고 수렴이 안정된다. 특히 SGD와 SGD+Momentum에서 과적합 방지 효과가 더 뚜렷하게 나타났다. Adam은 이미 내부적으로 적응형 학습률을 사용하므로 LR Decay 효과가 상대적으로 작다.

5) Gradient 흐름이 특정 Optimizer에서 사라지거나 급격히 변할 때, 어떤 문제가 발생하는가?

- Vanishing Gradients, Exploding Gradients의 원인 분석

SGD - Gradient Exploding 문제

SGD에서 학습률이 너무 크면 파라미터 업데이트량이 과도하게 커진다. 업데이트량 = 학습률 x Gradient

Gradient가 이미 큰 상황에서 학습률까지 크면 Loss가 최솟값을 넘어 반대 방향으로 튕겨나가는 Overshooting이 반복된다. 결국 Loss 값이 계속 증가하거나 NaN(계산 불능)이 된다. 해결 방법: 학습률을 줄이거나, Gradient Clipping 적용 (임계값 초과 시 Gradient를 잘라냄)

SGD - Vanishing Gradient 문제

SGD는 Gradient를 그대로 사용하기 때문에, 깊은 네트워크에서 역전파 시 Gradient가 레이어를 거칠수록 급셈으로 줄어든다. 초기 레이어에 도달하면 Gradient가 거의 0이 되어 해당 레이어의 가중치가 거의 업데이트되지 않는다. 결과적으로 깊은 레이어일수록 학습이 제대로 이루어지지 않는다.

Adam - Gradient Exploding 완화 구조

Adam의 업데이트 규칙에서 분모 항은 다음과 같다:

실제 업데이트량 = 학습률 $\times$ $m_hat / (\sqrt{v_hat} + \epsilon)$

여기서 $\epsilon = 1e-8$ (0.00000001)이 핵심 안전장치 역할을 한다. v_hat 이 매우 작아져 $\sqrt{v_hat}$ 이 거의 0이 되더라도 ϵ 이 분모를 0이 되지 않도록 막아준다. 따라서 업데이트량이 무한히 커지는 상황을 방지한다.

또한 v_hat 자체가 과거 Gradient 제곱의 평균이므로, Gradient가 자주 큰 파라미터는 분모가 커져 자동으로 업데이트량이 제한된다.

6) Optimizer가 동일한 네트워크에서 서로 다른 학습 패턴을 보이는 이유는?

- SGD, Adam, SGD+Momentum의 학습 곡선이 왜 다르게 움직이는지, 수식적 근거를 포함하여 설명하세요

SGD는 1차 Gradient만 사용하는 반면, Adam은 1차(m_t)와 2차(v_t) Moment를 모두 활용한다. SGD+Momentum은 Exponential Moving Average로 이전 Gradient 방향을 기억한다. 각 방법이 Loss Landscape를 탐색하는 전략이 다르기 때문에 동일 네트워크에서도 수렴 경로, 속도, 최종 수렴점이 달라진다. Adam이 더 복잡한 정보를 활용하므로 대부분의 경우 더 나은 성능을 보인다.

6. 결론 및 개선사항

Adam이 세 Optimizer 중 가장 빠르고 안정적인 수렴을 보이며 최고 정확도를 달성했다. SGD+Momentum은 SGD보다 명확히 개선되었으나 Adam에는 미치지 못했다. ExponentialLR Decay는 모든 Optimizer에서 후반부 수렴 안정성을 개선했으며 특히 SGD 계열에서 효과가 컸다. 개선 방안: AdamW(가중치 감쇠 포함), Cosine Annealing LR, 더 깊은 네트워크 구조 실험.